PHARMACY DBMS

From stock records to safer dispensing

A proposed relational database and desktop interface for a small, single-location pharmacy.



Made with GAMMA

Manual records make pharmacy control fragile

The risk begins when one medicine has several facts and several disconnected records.

Many variables

Medicines vary by manufacturer, batch, expiry date and selling price.

Disconnected records

Registers or spreadsheets make expiry checks, quantity tracking and prescription validation harder.

The proposed answer

One relational database with a Tkinter desktop interface.

Made with GAMMA

The system connects the pharmacy’s core workflows

Each stock-affecting action becomes part of one traceable path.

1
Receive stock
Into dated batches

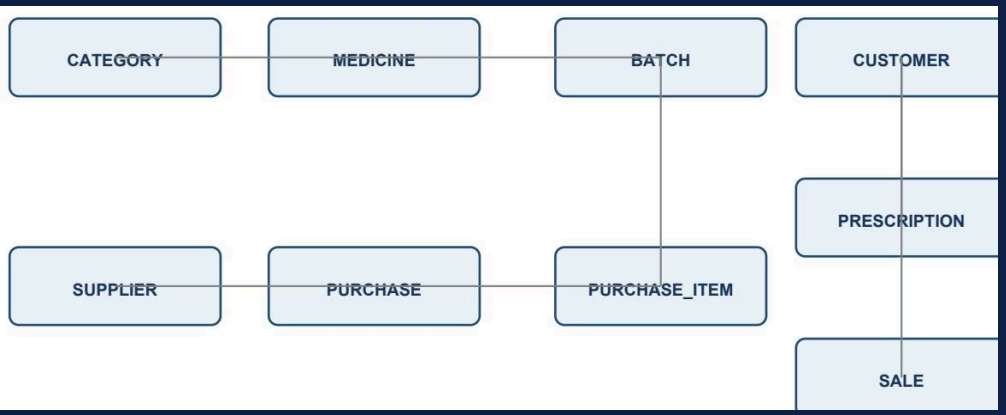
2
Search
Medicines and prescription requirements

3
Record
Prescriptions and sales

4
Trace
Reduce stock and preserve movement history

Designed for a small, single-location pharmacy; not a replacement for clinical judgement or statutory procedures.

Made with GAMMA



One schema links medicines, batches, people and transactions

Stock structure

Category → Medicine → Batch

Purchasing

Supplier → Purchase

Dispensing

Customer → Prescription and Sale

History

Stock movements provide history.

Made with GAMMA

Batch-level design makes expiry and stock visible

1

Medicine

Catalogue details, manufacturer, strength and prescription flag.

2

Batch

Code, expiry date, quantity, cost and selling price.

3

Transactions

Purchases, prescriptions, sales and stock movements.

4

Integrity

Primary keys, foreign keys, unique constraints and checks.

Made with GAMMA

Normalization reduces duplication and update anomalies

The report presents 3NF as the target design, subject to final implementation review.

1NF

One value per field; repeating items move into item tables.

2NF

Item details depend on their own key.

3NF

Supplier, category, customer and medicine details live in master tables.

Made with GAMMA

Python and SQL divide the work cleanly

The interface guides the workflow; the database preserves the records.

Tkinter / ttk

Forms, search fields, buttons and tabular results.

Python

Validation, workflow logic and user feedback.

SQL database

Persistent records, joins, aggregates and reports.

Transactions

Sale and stock deduction commit together or roll back together.

Made with GAMMA

Validation protects every stock-affecting action

Rules stop invalid data before it changes inventory or becomes difficult to trace.

1. **Required fields**

   Reject blank required fields and non-positive quantities.

2. **Expiry**

   Block sales from expired batches.

3. **Prescription**

   Require prescription details for controlled medicines.

4. **Stock level**

   Reject sales above available stock.

5. **Database safety**

   Use parameterized SQL and rollback on database errors.

Made with GAMMA

Testing comes before claiming the prototype works

The design identifies what must be checked; execution evidence is still required.

Planned check	Coverage	Status
Valid entry and search	Names, records and retrieval	To be tested
Invalid quantities	Blank names and negative quantities	To be tested
Inventory safeguards	Expired batches and insufficient stock	To be tested
Valid sales	Sale and stock deduction together	To be tested

Next: run the application, capture real screenshots, replace placeholders and verify the installed environment.

Made with GAMMA